Computação gráfica — pipeline clássico

A logo e-colab em OpenGL

Da janela ao tubo que desenha a pétala



Quatro cópias da mesma malha. Laranja em cima, verde embaixo. A diagonal que desce da esquerda para a direita é a maior, como na referência.

O miolo de cada pétala recua em z. De frente lê-se a logo. De lado, um funil.

A palavra da referência não entra neste protótipo. O programa desenha o símbolo.

Esta apostila explica o programa que está na pasta `examples`. A ordem é a de um quadro: primeiro a OpenGL como máquina de estados, depois a matriz que move o vértice, e só então a curva que vira tubo. Luz, tonalização e Z-buffer fecham o caminho até o pixel.

Os números são os do código. Quando um comentário do fonte e a conta discordam, vale a conta, e o texto diz onde isso acontece. O caso concreto é a normal da elipse: o comentário diz "para fora"; o vetor calculado aponta para o interior da curva. A normal que a luz enxerga é outra, e as duas coisas ficam separadas na seção da geometria.

Sumário

1 Como estudar com o programa aberto **1**

2 A cena **1**

3 Qual OpenGL é esta **2**

4 Um quadro, do clear ao swap **2**

5 A máquina de estados **3**

6 Do objeto à janela **4**

7 Coordenada homogênea **5**

8 Como a matriz mora na memória **5**

9 glMultMatrixf multiplica pela direita **6**

10 Translação, escala e rotação **6**

10.1 Translação e escala . . . 7

10.2 As três rotações . . . 7

10.3 O que a escala faz com a normal . . . 7

11 As quatro pétalas **8**

12 A câmera **9**

13 As duas projeções **10**

13.1 Perspectiva . . . 10

13.2 Ortográfica . . . 11

14 A geometria da pétala **12**

14.1 A elipse que também anda em z . . . 12

14.2 O triedro da seção . . . 12

14.3 O tubo . . . 13

14.4 Onde os anéis se sentam . . . 14

14.5 O funil . . . 15

14.6 A cor não é geometria . . . 16

14.7 O que cada constante muda . . . 17

15 Normal, Phong e Gouraud **17**

15.1 O modelo de Phong . . . 17

15.2 O que a OpenGL calcula . . . 17

15.3 Gouraud é outra decisão . . . 18

16 O Z-buffer **19**

17 A demonstração, tecla por tecla **19**

18 Erros que custam uma tarde **20**

19 Exercícios **21**

20 Mapa do código **22**

21 Glossário **23**

1 Como estudar com o programa aberto

A leitura acompanha o executável. Cada bloco abaixo termina numa tecla. Se a tecla não produz o que a seção descreveu, a seção ainda não foi entendida: a tela é o critério.

Tabela 1: O que ler e o que apertar em seguida.

Depois de	O que tem de estar claro	Tecla
seção 4	quem limpa, quem projeta, quem gira	abrir o programa
seção 11	$p' = R_z T S p$	`M`, depois `+`
seção 13	perspectiva muda o tamanho com a distância	`P` e `O`
seção 14	a pétala é tubo ao longo de elipse	`F`, `3`, `W`
seção 15	Phong é a conta; Gouraud é a interpolação	`L` e `G`
seção 16	sem teste, ganha quem foi desenhado por último	`Z`

No Linux, com FreeGLUT instalado:

terminal

```
sudo apt install build-essential cmake freeglut3-dev
cmake -S . -B build && cmake --build build
./build/ecolab_exemplo
```

A tecla `M` escreve no terminal que lançou o programa, não na janela. A barra de título mostra projeção, Gouraud, luz e Z-buffer.

Para dizer em voz alta

A cena abre inclinada de propósito. Vista frontal é a tecla `F` ou `1`. A referência está em `docs/referencia-ecolab.jpg`; o programa não carrega essa foto. A comparação é olho no olho, com a janela ao lado.

2 A cena

A referência é um símbolo plano: quatro lóbulos de traços concêntricos, laranja nos dois de cima, verde nos dois de baixo, a diagonal noroeste–sudeste um pouco maior, e a palavra e-colab por baixo. O protótipo fica com os quatro lóbulos e dá a eles volume. Cada lóbulo é a mesma malha, desenhada quatro vezes. Escala, distância ao centro e rotação mudam de uma cópia para a outra. A cor também.

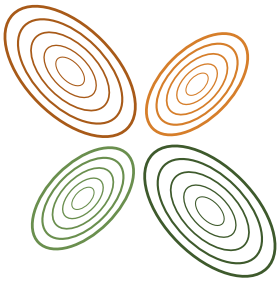


Figura 1: Leitura da cena. Cima, laranja; baixo, verde. Os lóbulos da diagonal principal são maiores e nascem um pouco mais longe do centro.

A malha nasce centrada na origem, comprida no eixo Y. Uma translação empurra esse centro ao longo de $+Y$. Uma rotação em Z leva esse $+Y$ para a diagonal. O miolo da elipse não fica no plano $z = 0$: ele recua até $z = -0{,}50$. Por isso a tecla `3`, quase de lado, mostra um funil em vez de um risco.

Esse volume é o que dá trabalho à luz e ao Z-buffer. Uma elipse desenhada com `GL_LINE` no plano não teria normal nem profundidade para discutir.

O plano de estudo do grupo, em `docs/PLANO.md`, ainda descreve a palavra desenhada por Bresenham ou DDA e uma placa atrás do símbolo. Esses arquivos não estão nesta árvore, e o `CMakeLists.txt` não os compila. Esta apostila cobre o que o executável faz hoje. Se a palavra voltar, o raster tem de ser código do grupo: uma lista de pixels, cada pixel um quadradinho. `glBegin(GL_LINES)` na palavra não cumpre esse item.

3 Qual OpenGL é esta

A OpenGL é uma interface de desenho. O programa descreve primitivas e estado; a implementação, dentro do driver, rasteriza. Há dois jeitos comuns de falar com ela.

No perfil moderno, a partir do OpenGL 3.3 core, o programa sobe os vértices para a memória da GPU, escreve um vertex shader que multiplica posição por matrizes, e um fragment shader que calcula a cor. `glBegin`, `glMatrixMode`, `glLight` e `glMultMatrix` não existem nesse perfil.

Este trabalho usa o outro jeito: o pipeline fixo, também chamado de modo imediato, via FreeGLUT. O vértice é emitido um a um entre `glBegin` e `glEnd`. A matriz entra por `glMultMatrixf`. A luz é um estado da máquina, não uma função escrita por vocês. A escolha não é estética. O enunciado pede a matriz de vocês aplicada com `glMultMatrix`. No perfil core essa chamada não existe.

As contas que a apostila desenvolve são as mesmas nos dois perfis. No shader moderno o vértice de recorte sai de

$$p_{\text{clip}} = P\,M\,p.$$

Aqui o driver faz esse produto com a matriz de projeção e a modelview que estiverem carregadas. Mudou quem multiplica. Não mudou a álgebra.

Três famílias de função ficam de fora da entrega, mesmo que a tela "funcione" com elas: `glTranslatef`, `glRotatef`, `glScalef`, `glOrtho`, `gluPerspective` e `gluLookAt`. Elas escondem a matriz. `glLoadIdentity`, `glPushMatrix`, `glPopMatrix` e `glMatrixMode` ficam: a pilha só guarda e restaura. Quem transforma o vértice é o arranjo que vocês montaram.

> **Cuidado**
>
> Uma chamada proibida que acerta a imagem zera o item da matriz ou o da projeção na hora da arguição. O teste visual não distingue `glRotatef` de `mat4_make_rotation_z`. O código distingue.

4 Um quadro, do clear ao swap

O FreeGLUT cria a janela e chama o programa de volta. `main` registra as funções e entra em `glutMainLoop`, que não retorna. Cada vez que a janela precisa ser redesenhada, o GLUT chama `display`. Mudar uma variável não redesenha sozinho: `glutPostRedisplay` avisa que o quadro está sujo.

examples/main.c

```
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH | GLUT_MULTISAMPLE);
glutDisplayFunc(display);
glutReshapeFunc(reshape);
glutKeyboardFunc(keyboard);
glutSpecialFunc(special_keys);
glutMouseFunc(mouse);
glutMotionFunc(motion);
glutMainLoop();
```

`GLUT_DOUBLE` pede dois buffers de cor. O desenho acontece no buffer de trás. `glutSwapBuffers`,

no fim de `display`, troca os dois. Sem isso o usuário veria a limpeza e a malha pela metade. `GLUT_RGB` pede cor em vermelho, verde e azul. `GLUT_DEPTH` cria o Z-buffer. Sem esse bit, `glEnable(GL_DEPTH_TEST)` não tem memória de profundidade e a tecla `Z` não tem o que desligar. `GLUT_MULTISAMPLE` pede antialiasing nas bordas do tubo. É conforto visual, não é o algoritmo de visibilidade do trabalho.

O corpo de `display` é o índice do resto da apostila.

1. Limpa cor e profundidade.
2. Acerta o viewport, o modo do polígono, a tonalização, o teste de profundidade e a luz, conforme as teclas.
3. `glMatrixMode(GL_PROJECTION)`, identidade, uma das duas projeções.
4. `glMatrixMode(GL_MODELVIEW)`, identidade.
5. Translação da câmera.
6. Posição da luz, ainda sem a rotação do objeto.
7. Rotação de pitch e yaw.
8. `draw_petals`, quatro vezes push, matriz, malha, pop.
9. Troca os buffers.

`glClear` não zera matriz. A identidade é chamada todo quadro, na projeção e na modelview. Sem ela, a rotação deste quadro compõe com a do quadro anterior e a cena foge.

examples/main.c

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glViewport(0, 0, g_width, g_height);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
/* perspectiva ou ortogonal, e apply_matrix(proj) */
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
/* câmera, luz, pitch, yaw, draw_petals */
glutSwapBuffers();
```

A cor de fundo é `glClearColor(0.97, 0.96, 0.94, 1)`, um branco quente. A profundidade limpa é `glClearDepth(1.0)`: o valor que o buffer trata como longe. O teste, armado em `init_gl`, é `GL_LESS`. Um fragmento pinta se a profundidade dele for menor que a armazenada.

`reshape` guarda a largura e a altura novas. Se a altura chegasse a zero, o aspect da projeção dividiria por zero; a função força altura mínima 1. O aspect é largura sobre altura. Na janela inicial, $960/720 = 4/3$.

5 A máquina de estados

A OpenGL deste programa não recebe a câmera como argumento. Ela consulta o que estiver valendo. Cor, normal, matriz ativa, teste de profundidade, modelo de tonalização e luz são estado. Dois erros de sintoma parecido nascem daí.

O primeiro: a matriz certa foi multiplicada no modo errado. `glMatrixMode(GL_PROJECTION)` e `glMatrixMode(GL_MODELVIEW)` escolhem qual das duas matrizes `glMultMatrixf` altera. Câmera aplicada com o modo ainda em projeção "não gira nada", porque o quadro seguinte dá `glLoadIdentity` na projeção e a modelview segue identidade.

O segundo: o estado foi armado cedo demais ou tarde demais. A luz é o exemplo. `glLightfv` lê a modelview corrente, transforma a posição e guarda o resultado em coordenada de olho. No quadro, isso acontece depois da translação da câmera e antes do yaw. A lâmpada fica parada enquanto a

logo gira, e o brilho caminha pela superfície. A mesma chamada depois do yaw gravaria a rotação dentro da lâmpada, e o brilho grudaria na pétala.

Estes são os estados que a arguição pode apontar na tela.

Tabela 2: Estado que muda a imagem, e a chamada que escreve nele.

Estado	Onde é escrito
Cor de limpeza e profundidade de limpeza	`glClearColor`, `glClearDepth`
Retângulo de pixels	`glViewport`
Polígono preenchido ou aramado	`glPolygonMode`
Cor por vértice ou cor por faceta	`glShadeModel`
Teste de profundidade	`glEnable/glDisable(GL_DEPTH_TEST)`
Luz ligada	`GL_LIGHTING`
Qual matriz `glMultMatrixf` altera	`glMatrixMode`
Modelview e projeção	`glLoadIdentity`, `apply_matrix`
Cópia da modelview	`glPushMatrix`, `glPopMatrix`
Cor do material	`glColor3fv`, com `GL_COLOR_MATERIAL`
Normal do vértice	`glNormal3f`

`glColor`, com a iluminação ligada, é ignorado até `glEnable(GL_COLOR_MATERIAL)`. Sem isso a logo inteira sai cinza e o grupo procura o bug na pétala. `setup_lighting` liga o estado e manda a cor alimentar o ambiente e a difusa do material, na frente e nas costas. O especular do material não vem do `glColor`; ele é fixo, declarado à parte.

6 Do objeto à janela

Um vértice da pétala passa por espaços sucessivos. Decorá-los evita a frase "a matriz da câmera" para três matrizes diferentes.

1. **Objeto.** A elipse está centrada na origem, comprida em Y. A normal do tubo também está nesse espaço. É o que `petal_draw` emite.
2. **Logo.** Escala, translação ao longo de Y e rotação em Z. As quatro pétalas passam a morar em volta da origem.
3. **Mundo virado.** Yaw em Y e pitch em X. A logo gira; a câmera, nesta implementação, não sai do lugar.
4. **Olho.** A translação $(0,\ 0{,}18,\ -d)$ coloca a origem da logo à frente da câmera. A câmera da OpenGL fica na origem e olha para $-Z$.
5. **Recorte.** A projeção produz a coordenada homogênea (x, y, z, w).
6. **NDC.** A divisão por w leva x, y e z ao cubo $[-1, 1]$.
7. **Janela.** O viewport espalha o quadrado $[-1, 1] \times [-1, 1]$ nos pixels. No mundo da OpenGL, y cresce para cima. No mouse do GLUT, y cresce para baixo.
8. **Fragmento.** Cada pixel coberto ganha cor e profundidade. O Z-buffer decide se essa profundidade substitui a que já estava lá.



Figura 2: Cinco nomes para não chamar tudo de "a matriz". A projeção P não é multiplicada por vocês com a modelview: o driver faz $P\,M\,p$.

Há dois sistemas destros, e os dois são verdade ao mesmo tempo. No mundo, antes da câmera, $+X$ aponta para a direita da tela, $+Y$ para cima e $+Z$ para quem olha. A câmera da OpenGL, porém, senta na origem e olha para o eixo Z negativo. Um ponto à frente da câmera tem z de olho negativo. A translação da câmera existe para fabricar esse z negativo: ela empurra a logo para $z = -d$.

Para dizer em voz alta

Ponto à frente da câmera tem z negativo no espaço do olho. O fundo do Z-buffer é 1, que corresponde ao lado longe do NDC. Perto é profundidade pequena. `GL_LESS` guarda o menor valor, isto é, quem está mais perto.

7 Coordenada homogênea

Translação não é transformação linear. $T(p) = p + t$ não passa pela origem, então não cabe numa matriz 3×3. A coordenada homogênea acrescenta uma quarta componente e devolve a translação ao mundo das matrizes.

Um ponto do espaço vira a coluna $(x, y, z, 1)$. Um vetor, que não tem posição, vira $(x, y, z, 0)$. A mesma matriz 4×4 age nos dois. Na translação, a última coluna é $(t_x, t_y, t_z, 1)$. Multiplicando um ponto, o 1 puxa t para o resultado. Multiplicando um vetor, o 0 anula t: direção não se translada. Normal é direção. Por isso a translação da pétala não empurra a normal, e ainda assim a rotação e a escala atuam nela.

A quarta linha da matriz de projeção perspectiva não é $(0, 0, 0, 1)$. Ela escreve $-z$ em w. Depois da matriz, a GPU divide x, y e z por w. Essa divisão é a perspectiva. `mat4_transform_point` não a faz. A tecla `M` mostra um ponto no espaço da logo, ainda sem projeção e sem câmera.

A luz usa a mesma distinção. A posição em `display` é (1,8, 2,4, 5,0, 1). O 1 final declara luz pontual: a OpenGL transforma esse valor como ponto. Se a quarta componente fosse 0, seria luz direcional, transformada como vetor, e os raios chegariam paralelos.

8 Como a matriz mora na memória

No papel a matriz se escreve por linhas. Na OpenGL ela se armazena por colunas. O elemento da linha r e da coluna c, com índices começando em zero, fica em

$$m[c \cdot 4 + r]. \tag{1}$$

A translação, que no papel está na última coluna, ocupa os índices 12, 13 e 14. Não ocupa o fim de cada linha.

a memória desce a coluna

coluna 0	coluna 1	coluna 2	translação
0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

Figura 3: Os dezesseis endereços. A coluna da translação está destacada. t_y, por exemplo, é $m[13]$.

`mat4_identity` zera o arranjo e escreve 1 em $m[0]$, $m[5]$, $m[10]$ e $m[15]$, a diagonal. `mat4_print` percorre linha por linha na hora de imprimir, lendo $m[c \cdot 4 + r]$, para o terminal mostrar a matriz do jeito do caderno. A memória por baixo continua por coluna.

A multiplicação segue a mesma convenção. `mat4_multiply(out, a, b)` calcula $a\,b$, com a coluna à direita.

examples/math3d.c · o elemento (linha, coluna)

```
r[col * 4 + row] =
    a[0 * 4 + row] * b[col * 4 + 0] +
    a[1 * 4 + row] * b[col * 4 + 1] +
    a[2 * 4 + row] * b[col * 4 + 2] +
    a[3 * 4 + row] * b[col * 4 + 3];
```

O índice `col * 4 + row` é a equação (1). O somatório é a regra de sempre: a linha de a contra a coluna de b. O resultado vai primeiro para um arranjo temporário, e só depois para `out`. Por isso `out` pode ser o próprio `a` ou o próprio `b` sem que a conta se pise.

examples/math3d.c · ponto coluna, com w = 1

```
*ox = m[0] * x + m[4] * y + m[8] * z + m[12];
*oy = m[1] * x + m[5] * y + m[9] * z + m[13];
*oz = m[2] * x + m[6] * y + m[10] * z + m[14];
```

Cada linha lê uma linha da matriz, descendo quatro colunas. É M vezes $(x, y, z, 1)$. A componente w do resultado não é calculada: para as matrizes afins desta cena ela continua 1.

Cuidado

Montar a matriz por linha e entregar esse vetor a `glMultMatrixf` coloca t_x em $m[3]$, que a OpenGL lê como o fim da primeira coluna, não como translação. A imagem entorta de um jeito que não se conserta chutando o ângulo.

9 glMultMatrixf multiplica pela direita

`apply_matrix` é a única porta das matrizes do grupo.

examples/math3d.c

```
void apply_matrix(const float m[16])
{
    glMultMatrixf(m);
}
```

`glMultMatrixf(A)` substitui a matriz corrente M por MA. O vértice é coluna à direita, então a última matriz aplicada é a primeira que toca o vértice. Depois de `glLoadIdentity`:

$$\texttt{apply}(A);\ \texttt{apply}(B);\ \texttt{apply}(C); \quad \Longleftrightarrow \quad p' = A\,B\,C\,p. \tag{2}$$

C age primeiro, B em seguida, A por último. A frase para levar à arguição é esta: a chamada de baixo, no fonte, é a transformação de dentro, na álgebra.

A pilha não entra nessa conta. `glPushMatrix` copia a modelview. `glPopMatrix` restaura a cópia. Nenhuma das duas gira vértice. Sem o pop, a segunda pétala herda $R_z T S$ da primeira e a rotação se acumula.

10 Translação, escala e rotação

As seis funções de `math3d.c` preenchem uma identidade e escrevem o bloco que muda. O ângulo entra em radianos. Em `main.c` os graus são convertidos na hora da chamada, multiplicando por $\pi/180$. Dentro de `petal.c` o parâmetro da elipse já nasce em radianos: uma volta é 2π.

10.1 Translação e escala

$$T(t) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \qquad S(s) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

No arranjo, t_x, t_y, t_z vão para $m[12]$, $m[13]$, $m[14]$. A escala vai para $m[0]$, $m[5]$, $m[10]$. Escala uniforme, o caso da pétala, usa o mesmo número nos três. Escala com $s_x \neq s_y$ mudaria a forma e também a direção da normal; a deste programa não faz isso.

10.2 As três rotações

A rotação positiva segue a mão direita: o polegar aponta no sentido do eixo, e os dedos fecham no sentido do ângulo positivo. Na tela, a rotação positiva em Z é anti-horária. É a que posiciona as pétalas.

$$R_z = \begin{pmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_y = \begin{pmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

com $c = \cos\theta$ e $s = \sin\theta$. A conferência com o fonte é pelo índice. Em `mat4_make_rotation_z`: $m[0] = c$, $m[1] = s$, $m[4] = -s$, $m[5] = c$. Quem escrever $m[1] = -s$ espelha a logo.

R_x gira Y na direção de Z. R_y gira Z na direção de X, e portanto leva um ponto em $+X$ para um z negativo. As duas aparecem na câmera, não na pétala.

10.3 O que a escala faz com a normal

A OpenGL transforma a normal pela inversa transposta do bloco 3×3 da modelview. A translação sai da conta, porque normal é vetor. A razão da inversa transposta é geométrica: a normal tem de continuar perpendicular à tangente depois da transformação.

Se uma tangente t vira At, a normal n' precisa cumprir $n' \cdot (At) = n \cdot t$ para toda tangente. Isso força $n' = (A^{-1})^{\top} n$.

Numa escala não uniforme a direção muda. Um caso pequeno basta. A tangente $(1, 1)$ tem normal $(-1, 1)$. Depois de $A = \mathrm{diag}(2, 1)$, a tangente vira $(2, 1)$. A normal transformada pela inversa transposta é $(-1/2,\ 1)$, e o produto escalar com $(2, 1)$ continua zero. A normal ingênua $An = (-2, 1)$ não é mais perpendicular.

Na pétala a escala é uniforme, $S = sI$. A inversa transposta vale $(1/s)I$: a direção sobrevive e o comprimento vira $1/s$. A malha já entrega normal de comprimento 1, feita por `normalize3`. A modelview desfaz esse comprimento sempre que $s \neq 1$. A pétala grande usa $s = 1{,}10$, a pequena usa $s = 0{,}84$, e a tecla + muda outro fator por cima. Sem uma renormalização, $N \cdot L$ sairia em escala errada: a pétala grande ficaria escura demais, a pequena clara demais, e o brilho especular mudaria de largura.

`glEnable(GL_NORMALIZE)` faz essa segunda normalização, já no espaço do olho, para toda normal que a modelview tocar. As duas normalizações têm empregos diferentes. `normalize3` acerta o tubo no espaço do objeto. `GL_NORMALIZE` desfaz o estrago da escala.

11 As quatro pétalas

examples/main.c · uma cópia

```
glPushMatrix();
mat4_make_rotation_z(rz, degrees[i] * MATH3D_PI / 180.f);
apply_matrix(rz);
mat4_make_translation(t, 0.f, out, 0.f);
apply_matrix(t);
mat4_make_scale(s, size, size, size);
apply_matrix(s);
petal_draw(/* laranja se i < 2, senão verde */);
glPopMatrix();
```

Pela equação (2),

$$p' = R_z(\theta)\, T(0, d, 0)\, S(s)\, p. \tag{3}$$

A escala acontece em volta da origem local da pétala. A translação empurra essa origem ao longo de $+Y$, por uma distância que a escala não altera. A rotação leva o eixo $+Y$ para a diagonal. Essa separação é o conteúdo da tecla +: os anéis crescem, o centro da pétala não se afasta. Com escala alta, o lóbulo avança sobre a origem e as quatro malhas se atravessam. É o comportamento certo desta ordem, não um acidente.

Os quatro ângulos e os dois tamanhos:

Tabela 3: As quatro chamadas de `petal_draw`. Ângulo positivo é anti-horário. "Grande" usa escala 1,10 e distância 0,44 × 1,04. "Pequena" usa 0,84 e 0,44 × 0,90.

i	θ	lugar	cor	tamanho
0	$-45°$	superior direito	laranja	pequeno
1	$+45°$	superior esquerdo	laranja	grande
2	$135°$	inferior esquerdo	verde	pequeno
3	$225°$	inferior direito	verde	grande

A conta do lugar: a translação produz o ponto $(0, d, 0)$, e R_z leva esse ponto a $(-d \sin\theta,\ d\cos\theta,\ 0)$. Para $\theta = +45°$, seno e cosseno valem $\sqrt{2}/2$, então o centro cai em x negativo e y positivo: superior esquerdo. Para $\theta = -45°$, o seno troca de sinal e o centro vai para o superior direito. A diagonal grande é a dos índices 1 e 3, noroeste e sudeste, que é a diagonal mais cheia da referência.

A elipse é comprida em Y e está centrada na própria origem. Metade dela aponta de volta para o centro da logo. No lóbulo grande, o centro fica a 0,458 da origem e a semi-altura, já escalada, vale cerca de 0,605. A ponta interna cai em $0{,}458 - 0{,}605 = -0{,}147$: atravessa a origem. A ponta externa vai a 1,06. No lóbulo pequeno a ponta interna fica em −0,066. O miolo da logo é zona de cruzamento de propósito. Quem pinta por cima, nessa zona, é decisão do Z-buffer.

Conta para conferir com a tecla M

Escala da tecla `M` igual a 1, translação $(0,\ 0{,}44,\ 0)$, rotação de 45°, ponto local $(0{,}20,\ 0,\ 0)$.
$Sp = (0{,}20,\ 0,\ 0)$.
$TSp = (0{,}20,\ 0{,}44,\ 0)$.
$c = s = 0{,}707107$.

$$x' = c \cdot 0{,}20 - s \cdot 0{,}44 = -0{,}1697,$$
$$y' = s \cdot 0{,}20 + c \cdot 0{,}44 = 0{,}4525.$$

O `printf` com três casas mostra `(-0.170, 0.453, 0.000)`. A matriz R_zTS, impressa por linha,

tem a translação já rotacionada na última coluna:

$$\begin{pmatrix} 0{,}707 & -0{,}707 & 0 & -0{,}311 \\ 0{,}707 & 0{,}707 & 0 & 0{,}311 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Com a escala em 1,2 o mesmo ponto cai em $(-0{,}141,\ 0{,}481,\ 0)$. A coluna da translação não muda: S entrou antes de T.

Cuidado

A tecla `M` não imprime a matriz de uma pétala da cena. Ela usa a escala global sozinha, a distância 0,44 sem o fator 1,04 ou 0,90, e o ângulo fixo de 45°. É a conta limpa do caderno. A pétala de +45° que está na tela ainda multiplica por 1,10 e afasta o centro até 0,458.

Vale a pena prever a ordem trocada antes de experimentar. Se as chamadas passarem a ser `apply(S)`, `apply(T)`, `apply(Rz)`, o produto vira $S\,T\,R_z$. A rotação age com a pétala ainda centrada, e girar em volta da origem não tira o centro do lugar. A translação seguinte manda as quatro cópias para o mesmo ponto $(0, d, 0)$. A escala, agora por fora, cresce em volta da origem do mundo e aumenta também a distância. As quatro pétalas empilham no eixo Y e a tecla + passa a afastá-las. A ordem do fonte não é convenção de escrita.

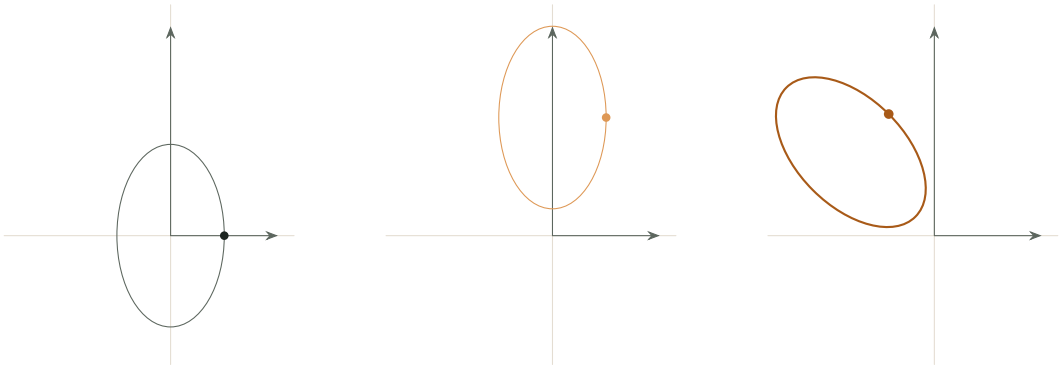


Figura 4: A mesma elipse, com escala 1, nos três tempos da equação (3). Os três quadros usam a mesma escala. O ponto é o $(0{,}20,\ 0)$ da tecla `M`: primeiro na ponta direita, depois em $(0{,}20,\ 0{,}44)$, por fim em $(-0{,}17,\ 0{,}45)$.

12 A câmera

Não há `gluLookAt`. A câmera é uma translação e duas rotações, as mesmas funções da pétala.

examples/main.c

```
mat4_make_translation(cam, 0.f, 0.18f, -g_cam_dist);
apply_matrix(cam);
glLightfv(GL_LIGHT0, GL_POSITION, light_pos);
mat4_make_rotation_x(rx, g_pitch * MATH3D_PI / 180.f);
apply_matrix(rx);
mat4_make_rotation_y(ry, g_yaw * MATH3D_PI / 180.f);
apply_matrix(ry);
```

O vértice da logo, já transformado pela equação (3), chega ao olho por

$$p_{\text{olho}} = T(0,\ 0{,}18,\ -d)\,R_x(\text{pitch})\,R_y(\text{yaw})\,p_{\text{logo}}. \tag{4}$$

Primeiro o yaw, em torno de Y. Depois o pitch, em torno de X. Por último o recuo. A câmera

permanece na origem do olho, olhando para $-Z$. O que se move é a logo. Para um objeto na origem, girar o objeto diante de uma câmera fixa tem o mesmo enquadramento que orbitar a câmera. A luz desfaz a ilusão: ela é gravada antes das rotações, então não gira junto. O par câmera–luz fica rígido, e a superfície da logo passa por baixo do brilho. É um prato giratório com lâmpada fixa, não uma câmera andando em volta de uma lâmpada pregada na parede.

O 0,18 em Y coloca a origem da logo um pouco acima do centro da janela. A distância d começa em 5,2, e a roda do mouse a varia entre 1,6 e 16. Aumentar d afasta. Na perspectiva a logo encolhe porque foi para mais longe. Na projeção ortográfica ela encolhe por outro motivo, tratado na seção seguinte: o retângulo do mundo cresce com d.

O yaw inicial é $-32°$ e o pitch inicial é $26°$. A vista de cima, tecla `4`, usa pitch $80°$. Isso confirma o sinal: pitch positivo traz o $+Y$ do modelo para o lado $+Z$, isto é, para perto da câmera, e a janela passa a mostrar a parte de cima. O pitch fica preso a $\pm 89°$. Em $\pm 90°$ o eixo do yaw alinha com a direção de visada e a vista vira de cabeça para baixo ao cruzar o polo.

Juntando as duas fórmulas, um vértice local da malha percorre

$$p_{\text{olho}} = T_{\text{cam}}\, R_x\, R_y\, R_z\, T\, S\, p. \tag{5}$$

A projeção não entra nesse produto. O driver, no pipeline fixo, completa

$$p_{\text{clip}} = P\, p_{\text{olho}}.$$

> **Para dizer em voz alta**
>
> A luz é declarada depois de T_{cam} e antes de R_x e R_y. Espaço ligado, a logo gira e o brilho anda. Se `glLightfv` escorregar para baixo das rotações, o brilho gruda na pétala e a demonstração de Phong perde o movimento.

13 As duas projeções

As duas matrizes são montadas em `math3d.c` e aplicadas depois de `glMatrixMode(GL_PROJECTION)` e `glLoadIdentity`. A tecla `P` escolhe a perspectiva. A tecla `O` escolhe a ortográfica. O código chama a segunda de ortogonal; o nome de livro é projeção ortográfica. São a mesma transformação: raios paralelos, sem ponto de fuga.

13.1 Perspectiva

O campo de visão vertical é $\varphi = 40°$. Seja a o aspect, largura sobre altura, e sejam n e f os planos perto e longe. No programa, $n = 0{,}1$ e $f = 80$.

$$f_v = \frac{1}{\tan(\varphi/2)}, \tag{6}$$

$$P = \begin{pmatrix} f_v/a & 0 & 0 & 0 \\ 0 & f_v & 0 & 0 \\ 0 & 0 & A & B \\ 0 & 0 & -1 & 0 \end{pmatrix} \qquad \begin{aligned} A &= \frac{f+n}{n-f}, \\ B &= \frac{2fn}{n-f}. \end{aligned} \tag{7}$$

No arranjo: $m[0] = f_v/a$, $m[5] = f_v$, $m[10] = A$, $m[11] = -1$, $m[14] = B$. O -1 está na última linha, terceira coluna. Ele copia $-z_{\text{olho}}$ para w.

Um ponto de olho $(x, y, z, 1)$, com $z < 0$, sai com $w' = -z$. A divisão perspectiva produz

$$x_{\text{ndc}} = \frac{f_v\, x}{a\,(-z)}, \qquad y_{\text{ndc}} = \frac{f_v\, y}{-z}.$$

Quanto mais longe, maior $-z$, menor o ponto na tela. Essa é a frase inteira da perspectiva. O tamanho na janela é uma razão entre o tamanho no mundo e a distância.

Na distância padrão $d = 5{,}2$, $\tan 20^\circ \approx 0{,}364$ e a meia-altura visível no plano da origem vale $5{,}2 \times 0{,}364 \approx 1{,}89$. A ponta externa da pétala grande fica perto de 1,06 do centro, e na diagonal isso projeta cerca de 0,75 em Y. Cabe, com folga, no campo de 40°.

Os valores numéricos de A e B, com $n = 0{,}1$ e $f = 80$, são $A \approx -1{,}003$ e $B \approx -0{,}200$. A linha de z não é uma escala uniforme. Depois da divisão por w, passos iguais de z no mundo viram passos desiguais no buffer. Perto da câmera a precisão é maior. Com $f/n = 800$ isso já seria um problema numa cena enorme; aqui o objeto inteiro mora numa faixa estreita de z e não há o que brigar com o buffer.

Trocar φ para 20°, sem mexer em d, troca $\tan 20^\circ$ por $\tan 10^\circ \approx 0{,}176$. A meia-altura visível cai para cerca de 0,92. A logo ocupa bem mais da janela e continua dentro do quadro. É o exercício clássico: campo menor, objeto maior, e a câmera só precisa recuar se a ponta passar de ± 1 em NDC.

13.2 Ortográfica

$$O = \begin{pmatrix} \dfrac{2}{r-l} & 0 & 0 & -\dfrac{r+l}{r-l} \\ 0 & \dfrac{2}{t-b} & 0 & -\dfrac{t+b}{t-b} \\ 0 & 0 & \dfrac{-2}{f-n} & -\dfrac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{8}$$

A última linha é $(0, 0, 0, 1)$. Então w continua 1, a divisão perspectiva não altera tamanho, e um objeto longe sai com o mesmo tamanho de um objeto perto. Retas paralelas continuam paralelas. Não há ponto de fuga.

A chamada usa um retângulo centrado, de meia-altura $h = 0{,}45\,d$, meia-largura $h \cdot a$, e a faixa de profundidade de -20 a 20:

$$\texttt{mat4_make_ortho}(-ha,\ ha,\ -h,\ h,\ -20,\ 20).$$

Com $d = 5{,}2$, $h = 2{,}34$. O x e o y do centro somem, porque $r = -l$ e $t = -b$: a translação da projeção é zero e a origem do olho cai no centro da janela. A faixa $[-20, 20]$ é larga de propósito. O parâmetro perto negativo é legal nesta fórmula; ele só diz que um dos planos fica atrás do olho. A logo, com d entre 1,6 e 16, permanece dentro da fatia, então o Z-buffer ainda recebe uma ordem de profundidade.

Na mesma pose, as duas projeções não enquadram igual. A perspectiva, a 5,2 de distância, mostra cerca de $\pm 1{,}89$ em Y. A ortográfica mostra $\pm 2{,}34$. A logo aparece um pouco menor com a tecla `0`. O fator 0,45 não foi escolhido para igualar os dois quadros. Ele faz a roda do mouse valer também na ortográfica: aumentar d aumenta o retângulo, e a logo encolhe. O gesto de zoom é o mesmo. O mecanismo, não.

> **Para dizer em voz alta**
>
> Perspectiva: $w = -z$, e o longe encolhe. Ortográfica: $w = 1$, e o longe conserva o tamanho. As duas fotos da arguição saem da mesma pose. Quem mexe na seta entre um `P` e um `O` estraga a comparação.

No Windows os nomes `near` e `far` são macros vazias em `windows.h`. Os parâmetros do fonte se chamam `znear` e `zfar`. `gl_platform.h` existe para incluir o cabeçalho do Windows sem deixar essas macros comerem a matemática.

14 A geometria da pétala

A pétala não é uma superfície fechada com miolo preenchido. É um feixe de traços, e cada traço tem volume: um tubo fino ao longo de uma elipse. Volume dá normal. Normal dá o que a luz calcular.

Quatro inteiros, no topo de `petal.c`, mandam na malha.

Tabela 4: Resolução da malha. Mexer num deles muda uma coisa só; a tabela da seção 14.7 diz qual.

Nome	Valor	Papel
`PETAL_RINGS`	20	traços do miolo até a borda, em cada mecha
`PETAL_STRANDS`	3	famílias de elipses, com expoentes diferentes
`PETAL_SEGMENTS`	96	amostras de uma volta
`PETAL_SLICES`	8	lados da seção do tubo

São $20 \times 3 = 60$ tubos por pétala, quatro pétalas, cada tubo com oito fitas. O modo imediato emite isso de novo a cada quadro. Para uma logo que cabe na tela, o custo é irrelevante. O que importa é ler a malha.

14.1 A elipse que também anda em z

`ellipse_frame` devolve um ponto da curva e dois eixos da seção.

examples/petal.c · ponto e tangente

```
*px = rx * c;
*py = ry * s;
*pz = z0 + (z1 - z0) * (t / (2.f * MATH3D_PI));
float tx = -rx * s;
float ty = ry * c;
float tz = (z1 - z0) / (2.f * MATH3D_PI);
```

Em fórmula, com t de 0 a 2π,

$$\begin{aligned} p(t) &= (r_x \cos t,\ r_y \sin t,\ z_0 + (z_1 - z_0)\, t/(2\pi)), \\ T_{\text{cru}} &= (-r_x \sin t,\ r_y \cos t,\ (z_1 - z_0)/(2\pi)). \end{aligned} \tag{9}$$

O ponto (p_x, p_y) é a elipse de sempre. O z sobe linearmente com t: numa volta, a curva vai de z_0 a z_1. De frente, o observador olha contra Z e vê uma elipse. De lado, vê a rampa. A tangente é a derivada, e em seguida `normalize3` a põe com comprimento 1. Se o vetor fosse nulo, a função devolveria $(0, 0, 1)$ para não produzir um NaN lá na frente, na divisão pelo comprimento.

$t = 0$ é o ponto direito da elipse, $(r_x, 0)$. A tangente no plano XY, antes do z, aponta para $+Y$. O parâmetro cresce no sentido anti-horário.

14.2 O triedro da seção

O tubo precisa de um plano perpendicular à tangente, e de dois eixos nesse plano, para apoiar o círculo da seção.

examples/petal.c · normal de seção e binormal

```
*nx = -ty;
*ny = tx;
*nz = 0.f;
normalize3(nx, ny, nz);
*bx = ty * (*nz) - tz * (*ny);
*by = tz * (*nx) - tx * (*nz);
*bz = tx * (*ny) - ty * (*nx);
normalize3(bx, by, bz);
```

A tangente já está normalizada quando essas linhas rodam. O vetor N nasce de girar a parte XY da tangente 90° no sentido anti-horário, com componente z nula:

$$N_{\text{cru}} = (-\hat{T}_y,\ \hat{T}_x,\ 0), \qquad \hat{N} = N_{\text{cru}} / \|N_{\text{cru}}\|. \tag{10}$$

No ponto direito, $\hat{T} \approx (0,1,0)$ e $\hat{N} \approx (-1,0,0)$. A normal aponta para a origem da elipse, para o interior, não para fora. O comentário do fonte diz o contrário. A conta é a equação (10). Para uma curva percorrida no sentido anti-horário, girar a tangente 90° anti-horário aponta para a esquerda de quem caminha na curva, e a esquerda é o lado de dentro.

Isso não estraga o tubo. N aqui não é a normal de iluminação. É um eixo da seção. A binormal completa um triedro destro:

$$\hat{B} = \hat{T} \times \hat{N}. \tag{11}$$

O produto do fonte é exatamente esse, componente a componente. Como N foi construído perpendicular a T — o produto $\hat{T}_x(-\hat{T}_y) + \hat{T}_y\hat{T}_x$ se anula, e $N_z = 0$ — o par N, B forma uma base do plano normal à curva. No ponto direito, com a rampa de z ainda pequena, B aponta para $+Z$, para o lado da câmera.

Quem já viu o triedro de Frenet no cálculo reconhece as três letras e o produto vetorial. A normal daqui não é a normal principal de curvatura da curva no espaço. É a normal da elipse no plano XY, reaproveitada como eixo. Com a rampa de z pequena, as duas ficam próximas. Quando a rampa é íngreme, deixam de coincidir, e o tubo continua bem definido porque $B = T \times N$ mantém a seção perpendicular a T.

14.3 O tubo

examples/petal.c

```
float c = cosf(angle);
float s = sinf(angle);
float ox = c * nx + s * bx;
float oy = c * ny + s * by;
float oz = c * nz + s * bz;
glNormal3f(ox, oy, oz);
glVertex3f(px + radius * ox, py + radius * oy, pz + radius * oz);
```

$$o(\alpha) = \cos\alpha\ \hat{N} + \sin\alpha\ \hat{B}, \qquad v = p + \rho\, o(\alpha), \qquad n = o(\alpha). \tag{12}$$

α dá a volta na seção. ρ é `PETAL_TUBE_RADIUS`, 0,0082. O vértice está a ρ do eixo da curva, na direção o. A normal enviada à OpenGL é a mesma direção. Ela aponta para fora do fio, em todos os lados: para o interior da elipse quando $\alpha = 0$, para o exterior quando $\alpha = \pi$, e para os lados conforme B. É essa normal que entra em $N \cdot L$.

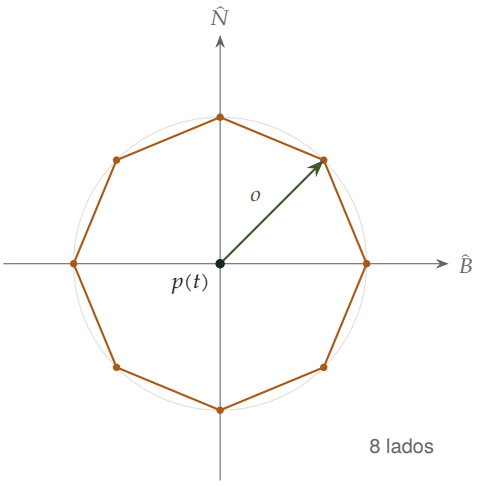


Figura 5: Seção do tubo, olhando ao longo da tangente. $\alpha = 0$ fica sobre $\hat{N}$. O octógono é o que a tecla `G`, com a luz ligada, mostra facetado. A normal de cada vértice é o raio.

Oito lados é uma escolha didática. Com mais lados o tubo fica redondo também em `GL_FLAT`, e a tecla `G` perde a demonstração. Com menos, o traço da logo deixa de parecer fio.

Cada lado vira uma triangle strip ao longo da volta. Para a fatia de ângulos α_0 e α_1, o laço de $i = 0$ até 96 inclusive emite, em todo t, o vértice de α_0 e o de α_1. O índice vai até `PETAL_SEGMENTS` para t alcançar 2π e a curva fechar: o último ponto repete o primeiro. A última fatia fecha o octógono pelo mesmo motivo, periodicidade do seno. Há vértice repetido na costura. No modo imediato isso não tem preço.

A ordem da fita concorda com a normal radial. No primeiro triângulo, as duas arestas que saem do vértice $\alpha = 0$ apontam aproximadamente para B e para T. O produto $B \times T$ devolve N, que é exatamente $o(0)$. Não é uma coincidência a corrigir no relatório. É a razão de a luz pegar o lado de fora do fio.

14.4 Onde os anéis se sentam

Os vinte anéis de uma mecha não são espaçados por igual. Um expoente maior que 1 em u^e segura os primeiros anéis perto do miolo. Sozinho, ele abre um vão grande antes da borda. A função `ring_position` devolve um pedaço desse vão no meio do caminho.

examples/petal.c

```
mid = sinf(u * MATH3D_PI);
mid = mid * mid;
return powf(u, exponent) * (1.f - 0.62f * mid) + u * (0.62f * mid);
```

$$m(u) = \sin^2(\pi u), \qquad v(u) = u^e\,(1 - 0{,}62\,m(u)) + u\,(0{,}62\,m(u)). \tag{13}$$

m vale 0 no miolo e na borda, e 1 no meio. Nas pontas, $v = u^e$. No meio, 62% do valor vem do espaçamento linear e o resto do expoente. Para $e = 2{,}15$ e $u = 1/2$, $v \approx 0{,}396$, enquanto a reta daria 0,5 e a potência pura daria 0,25.

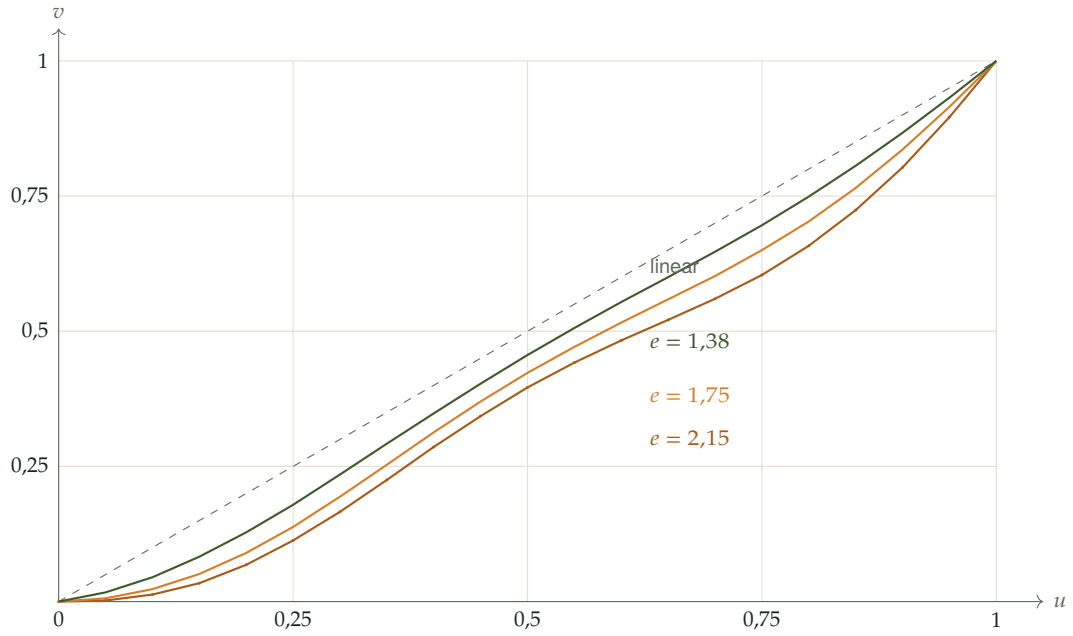


Figura 6: Raio normalizado v contra o parâmetro u. As três curvas são as três mechas. Os pontos são os vinte anéis da mecha de expoente 2,15: apertados no miolo, mais folgados na borda.

Cada mecha desloca a amostra. Com deslocamento $\delta \in \{0,\ 1/3,\ 2/3\}$ e vinte anéis,

$$u_0 = (k + \delta)/20, \qquad u_1 = \min\,(1,\ (k + 1 + \delta)/20),$$

e os raios usados são $v(u_0)$ e $v(u_1)$. Os expoentes são 2,15, 1,75 e 1,38. A mecha de expoente maior fica mais tempo perto do miolo. Os deslocamentos impedem que as três famílias caiam nas mesmas elipses. Há ainda um piso: se $v_1 < v_0 + 0{,}006$, v_1 é empurrado. Sem isso, a potência achata tanto o primeiro passo que o tubo nasce com comprimento nulo e a malha degenera.

O anel de índice k não interpola o raio ao longo da volta. Ele usa um raio só, o de v_0, e gasta a volta para subir de $z(v_0)$ até $z(v_1)$:

$$\begin{aligned} r_x(v) &= 0{,}058 + 0{,}29\,v, \\ r_y(v) &= 0{,}090 + 0{,}46\,v, \\ z(v) &= -0{,}50\,(1 - v)^{0{,}72} + \phi. \end{aligned} \tag{14}$$

Na borda, $v = 1$, a elipse mede 0,348 por 0,550. No miolo, 0,058 por 0,090. A razão r_y/r_x fica por volta de 1,55 nos dois extremos: a família é alongada no eixo da pétala do começo ao fim.

$\phi = (\delta - 0{,}5) \cdot 0{,}06$ é um empurrão minúsculo em z, entre $-0{,}03$ e 0,01. Perto da profundidade de 0,50, não separa as mechas. Inclina uma em relação à outra, e de lado os fios se cruzam em vez de coincidir.

14.5 O funil

$z(0) = -0{,}50$ e $z(1) = 0$. O miolo está mais longe da câmera; a borda, mais perto. O expoente 0,72, menor que 1, segura o meio da pétala para trás: em $v = 1/2$, $z \approx -0{,}304$, enquanto uma reta já estaria em $-0{,}25$. A tigela é funda no miolo e vem à frente mais depressa perto da borda.

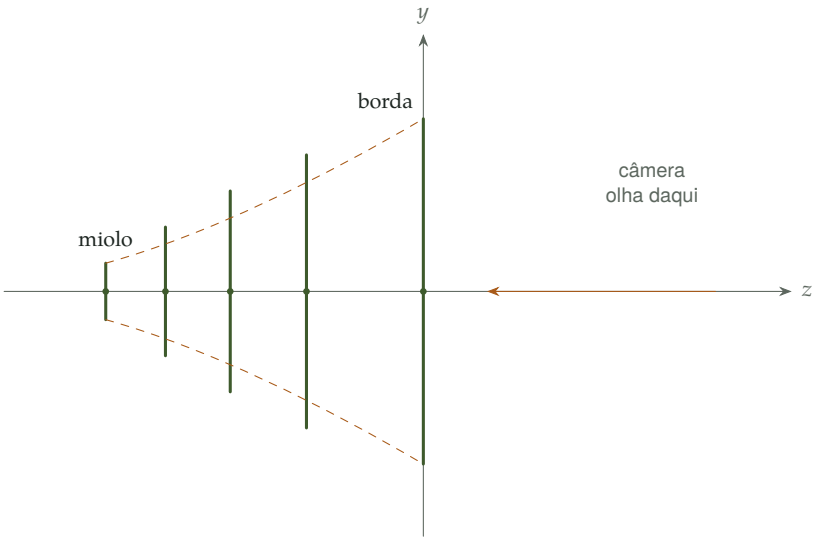


Figura 7: Uma pétala no próprio espaço, vista de lado. Cada traço vertical é a altura $2r_y$ de um anel, no z da equação (14). O largo está perto da câmera. A tecla `3` mostra as quatro cópias desta figura, já colocadas nas diagonais.

O número 0,72 não sai de uma fórmula da logo. É forma. A vista de lado é o lugar de julgá-lo. O mesmo vale para 0,50, a profundidade total: mais fundo, mais volume; fundo demais, e a vista frontal começa a denunciar a rampa.

14.6 A cor não é geometria

examples/petal.c

```
out[i] = dark[i] + (light[i] - dark[i]) * u;
```

u aqui é o v do anel, não o ângulo da elipse. Um tubo inteiro tem uma cor só. O degradê da logo é discreto, de tubo em tubo, do escuro no miolo ao claro na borda. As três mechas se intercalam, e o olho lê um degradê mais fino do que vinte amostras.

A cor entra por `glColor3fv`, em componentes de 0 a 1. Os literais do fonte são as amostras de 0 a 255 divididas por 255, tiradas da referência.

Tabela 5: As quatro cores. Com a luz ligada elas viram k_a e k_d, não a cor final do pixel.

Papel	RGB em 0..255	Uso
laranja escuro	215, 126, 42	miolo das pétalas de cima
laranja claro	234, 168, 85	borda das pétalas de cima
verde escuro	96, 134, 72	miolo das pétalas de baixo
verde claro	150, 186, 118	borda das pétalas de baixo

Há, portanto, duas variações em cada fio, e elas não se confundem. Ao longo da família de anéis, muda o material. Em volta de um mesmo fio, o material é constante e muda a normal. A redondez do traço é iluminação. O clarear para a borda é cor. Desligar a luz, tecla `L`, conserva o degradê e apaga a redondez.

14.7 O que cada constante muda

Tabela 6: Constantes de forma. O valor é o do fonte. A terceira coluna é o que olhar depois de mudar.

Constante	Valor	Se aumentar
`PETAL_RINGS`	20	mais traços; se colarem, desça o raio do tubo
`PETAL_TUBE_RADIUS`	0,0082	fio mais grosso
`PETAL_DEPTH`	0,50	funil mais fundo, vista `3`
expoente de z	0,72	o meio da pétala vem à frente mais cedo
expoentes das mechas	2,15; 1,75; 1,38	o miolo fica mais apertado
r_x, r_y	eq. (14)	lóbulo mais largo ou mais comprido
`PETAL_OUTWARD`	0,44	afasta o centro do lóbulo da origem
escalas 1,10 e 0,84		acentuam a diagonal desigual
fatores 1,04 e 0,90		afastam o lóbulo grande e aproximam o pequeno
`PETAL_SEGMENTS`	96	elipse mais lisa
`PETAL_SLICES`	8	fio mais redondo; o `GL_FLAT` fica menos legível

Para dizer em voz alta

Uma pétala, em uma frase: sessenta tubos elípticos, três famílias com expoentes diferentes, miolo em $z = -0{,}5$, borda em $z = 0$, normal de iluminação igual ao raio do tubo. `main.c` só copia essa malha quatro vezes.

15 Normal, Phong e Gouraud

15.1 O modelo de Phong

Num ponto da superfície, com normal unitária N, a direção unitária L do ponto até a luz, e a direção unitária V do ponto até o observador, o modelo de Phong escreve três parcelas:

$$I = I_a k_a + I_d k_d \max(N \cdot L,\ 0) + I_s k_s \max(R \cdot V,\ 0)^n, \tag{15}$$

$$R = 2(N \cdot L)\, N - L. \tag{16}$$

I_a, I_d, I_s são as intensidades ambiente, difusa e especular da luz. k_a, k_d, k_s são as reflectâncias do material. n é o brilho: grande, o especular fica um ponto; pequeno, fica uma mancha.

A parcela ambiente não depende da normal. Ela existe para o lado que a luz não alcança não ficar preto puro. A difusa é a lei do cosseno: superfície de frente para a luz, $N \cdot L$ perto de 1, fica clara; de perfil, fica escura; de costas, o máximo com zero a apaga. A especular depende de o reflexo R apontar para o observador.

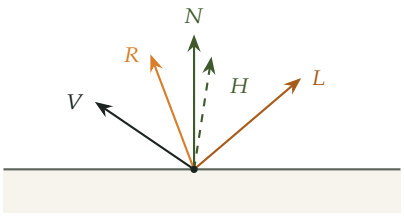


Figura 8: R é o reflexo de L em torno de N. H é o meio do caminho entre L e V, que é o vetor que a OpenGL usa no lugar de R.

15.2 O que a OpenGL calcula

O pipeline fixo não avalia $R \cdot V$. Ele avalia o modelo de Blinn–Phong. O vetor do meio é

$$H = \frac{L + V}{\|L + V\|}, \qquad \text{especular} = \max(N \cdot H,\ 0)^n. \tag{17}$$

H aponta para onde a normal precisa apontar para que a luz rebata no observador. Quando N coincide com H, o especular é máximo. O enunciado aceita a rotina da OpenGL. No relatório, a fórmula escrita é a (15), com uma frase dizendo que a implementação usa o halfway.

Há três guardas que a fórmula de quadro não mostra e a rotina aplica. Se $N \cdot L < 0$, a luz está do outro lado da superfície e o especular também vai a zero, não só a difusa. A atenuação com a distância não foi configurada: os coeficientes ficam no padrão, constante 1 e linear e quadrático 0, então a lâmpada não enfraquece com a distância. O que muda o brilho de um fio para o outro é o ângulo, não o fato de uma pétala estar mais longe da lâmpada. E `GL_LIGHT_MODEL_LOCAL_VIEWER` ligado faz V ir do vértice até a câmera, em vez de usar a direção fixa $(0, 0, 1)$.

`GL_SHININESS` é o n. O valor no fonte é 48, dentro da faixa 0 a 128 do pipeline fixo.

examples/main.c · os coeficientes

```
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient); /* 0.46, 0.46, 0.44 */
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient);              /* 0.22 */
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse);              /* 0.88 */
glLightfv(GL_LIGHT0, GL_SPECULAR, specular);            /* 0.28 */
glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, mat_specular); /* 0.22 */
glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 48.f);
glLightModeli(GL_LIGHT_MODEL_COLOR_CONTROL, GL_SEPARATE_SPECULAR_COLOR);
```

k_a e k_d não estão nessa lista. Eles vêm da cor da pétala, porque `GL_COLOR_MATERIAL` está ligado em `GL_AMBIENT_AND_DIFFUSE`. k_s é o cinza 0,22 do material, igual nas quatro pétalas. A luz especular é fraca de propósito, 0,28: um fio fino com especular branco forte vira um risco de giz e a cor da logo some.

`GL_SEPARATE_SPECULAR_COLOR` interpola o especular à parte e soma depois da cor difusa. Sem isso, o especular é somado à cor ainda no vértice e o resultado é grampeado em 1 antes de viajar pelo triângulo. Num laranja que já está alto, o brilho não tem onde aparecer e a parcela de Phong fica invisível na apresentação. O controle não transforma Gouraud em Phong por pixel. O brilho continua calculado nos vértices.

`GL_LIGHT_MODEL_TWO_SIDE` liga a equação também na face de trás, invertendo a normal. O material de trás é o mesmo. A fita já nasce coerente com a normal; o estado de dois lados cobre o que ainda assim aparecer virado, em especial na silhueta do tubo.

A posição da luz é reenviada todo quadro. Guardá-la uma vez no `init`, com a modelview ainda identidade, grava a lâmpada em coordenada de olho sem a translação da câmera. O ponto $(1{,}8,\ 2{,}4,\ 5)$ ficaria atrás do observador, porque z positivo é o lado de trás da câmera. O fonte repete a chamada no lugar certo e essa armadilha fica desarmada.

15.3 Gouraud é outra decisão

O modelo de Phong diz como I sai de N, L e V. A tonalização diz onde essa conta é feita, e o que se interpola no meio do triângulo.

Tabela 7: Três nomes que a arguição troca, e só um deles está na tecla `G`.

Nome	Onde a conta de luz acontece	O que anda pelo triângulo
Gouraud, `GL_SMOOTH`	em cada vértice	a cor já calculada
Constante, `GL_FLAT`	num vértice da faceta	uma cor só
Phong por pixel	em cada fragmento	a normal, e a luz é refeita

A tecla `G` alterna as duas primeiras. A terceira, Phong shading, o enunciado não pede e o pipeline fixo não faz. Phong, nesta entrega, é o modelo. Gouraud é o transporte da cor.

Com `GL_FLAT`, cada triângulo da fita usa uma normal só, a do vértice provocador. Os oito lados do tubo aparecem como facetas. Com `GL_SMOOTH`, as cores dos vértices — já iluminadas com normais

diferentes — são interpoladas, e o octógono parece redondo. A diferença só aparece com a luz ligada. Com a luz desligada, todos os vértices de um anel têm o mesmo `glColor`, e interpolar uma constante não muda nada. Quem aperta `G` no escuro e não vê efeito não quebrou a tecla.

O aramado, tecla `W`, é `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)`. Ele mostra a fita e a seção mesmo sem luz. É o jeito de confirmar que o traço tem oito lados e não é uma fita chata.

> **Para dizer em voz alta**
>
> Frase pronta: a luz avalia Phong–Blinn no vértice; `GL_SMOOTH` interpola essa cor; `GL_FLAT` repete a cor da faceta; sombrear Phong de verdade interpolaria a normal e refaria a conta no pixel, e não é o que a tecla `G` faz.

16 O Z-buffer

Cada pixel do framebuffer guarda, além da cor, uma profundidade. O teste desta cena é o mais simples:

1. A limpeza enche o buffer com 1, o longe.
2. Para cada fragmento, compara-se a profundidade dele com a armazenada.
3. Se a nova for menor (`GL_LESS`), a cor substitui a antiga e a profundidade também.
4. Se não for, o fragmento é descartado. A cor que já estava lá permanece.

Isso é `glEnable(GL_DEPTH_TEST)`, `glDepthFunc(GL_LESS)` e o bit `GL_DEPTH_BUFFER_BIT` na limpeza. A tecla `Z` só liga e desliga o teste. A função de comparação continua `GL_LESS`. Profundidades iguais não passam: o primeiro fragmento permanece.

A ordem de desenho é a do laço: pétala 0, depois 1, 2 e 3. Com o teste ligado, a ordem quase não importa. Ganha quem está mais perto da câmera. Com o teste desligado, ganha quem foi desenhado por último, mesmo que esteja atrás. Onde as malhas se cruzam — e elas se cruzam, porque a ponta interna atravessa a origem — a diferença fica óbvia. Na vista frontal o efeito existe, mas é mais frio, porque um lóbulo não tapa o outro inteiro. A foto que convence é a vista inclinada, lado a lado, `Z` ligado e `Z` desligado, sem mexer no ângulo entre as duas.

`glPushMatrix` não é visibilidade. Back-face culling, `GL_CULL_FACE`, também não entra: com uma normal inconsistente ele apaga um pedaço de tubo e atrapalha mais do que ajuda. O item é o buffer de profundidade.

Dois esquecimentos deixam o teste "ligado" e a imagem errada. Sem `GLUT_DEPTH` na criação da janela, não há buffer. Sem o bit de profundidade no `glClear`, o quadro anterior continua morando no buffer, e a cena em movimento mancha ou fura. Limpar a profundidade com 0, em vez de 1, também esvazia a imagem: nenhum fragmento tem profundidade menor que zero, e `GL_LESS` rejeita todos.

> **Para dizer em voz alta**
>
> O Z-buffer guarda quem está na frente. Ele não ordena os objetos antes de desenhar, não conhece a pétala e não substitui a matriz. Compara um número por pixel, depois que a projeção já transformou z.

17 A demonstração, tecla por tecla

A janela abre em yaw $-32°$ e pitch $26°$, distância 5,2, perspectiva, luz ligada, Gouraud, Z-buffer ligado, escala de pétala 1. Essa é a vista em que o volume aparece. A ordem abaixo cabe em cinco minutos e percorre os itens do trabalho.

1. `F`. Quatro lóbulos, laranja em cima, verde embaixo, a diagonal principal maior. Modelagem.

2. Setas, ou o arrasto com o botão esquerdo. Depois `W`. A seção do tubo aparece. `W` de novo volta ao preenchido.
3. `+` duas vezes e `M` no terminal, com a conta da seção das pétalas ao lado. A escala mudou o ponto; a translação da matriz impressa, não.
4. `P` e `O` sem tocar nas setas. Ortográfica um pouco menor, paralelas conservadas.
5. `L` apaga a redondez e deixa o degradê. `L` de novo. `G` facetá o fio. Uma frase: Phong é a conta, Gouraud é a interpolação.
6. Espaço. O brilho caminha porque a luz foi declarada antes da rotação.
7. Inclinar de novo e alternar `Z` no mesmo enquadramento.

Tabela 8: Todas as entradas que `keyboard`, `special_keys` e o mouse reconhecem.

Entrada	Efeito
arrasto, setas	yaw e pitch; seta soma 4°, arrasto soma 0,35° por pixel
roda, `[ ]`, Page Up/Down	distância da câmera; aumentar a distância afasta
`1`, `F`	frontal, yaw 0, pitch 0, conserva o zoom
`2`	a vista inicial, −32° e 26°
`3`	lado, yaw 90°, pitch 8°: o funil
`4`	de cima, pitch 80°
`R`	volta à vista inicial e à distância 5,2
`P`, `O`	perspectiva, ortográfica
`W`	aramado ou preenchido
`G`	`GL_SMOOTH` ou `GL_FLAT`
`L`	luz ligada ou cor direta
`Z`	teste de profundidade
`+`, `-`	escala da pétala, passo 0,05, presa a $[0{,}3,\ 2{,}2]$
espaço	gira o yaw sozinho; para ao escolher uma vista
`M`	imprime S, T, R_z e o ponto $(0{,}20,\ 0,\ 0)$
Esc	encerra

O arrasto soma y de tela ao pitch. No GLUT, y cresce para baixo, então arrastar para baixo aumenta o pitch e revela a parte de cima, coerente com a tecla `4`. A rotação automática não usa relógio: cada chamada ociosa soma 0,35° ao yaw. A velocidade depende de quantas vezes o GLUT entrega o ócio, não de um passo de tempo fixo.

18 Erros que custam uma tarde

1. Matriz preenchida por linha e passada a `glMultMatrixf`. A translação cai nos índices 3, 7 e 11.
2. Ordem dos `apply_matrix` invertida, e o ângulo alterado até "parecer" certo. A tecla `M` desmente.
3. `glMatrixMode` deixado em `GL_PROJECTION`. A câmera é apagada no quadro seguinte pela identidade da projeção.
4. `glRotatef` ou `gluPerspective` na pressa. A tela acerta e o item zera.
5. `glLoadIdentity` ausente. As transformações acumulam e a logo sai do quadro em poucos segundos.
6. Pop faltando. A segunda pétala herda a rotação da primeira.
7. Luz configurada só no `init`. Ela gruda na câmera, e no lugar errado.
8. `glColor` com iluminação ligada e `GL_COLOR_MATERIAL` desligado. A logo fica cinza.
9. Normal ausente. Com a luz ligada o tubo fica chapado ou preto. O aramado ainda mostra a seção; a luz, não.
10. Escala alta na ordem certa. Os lóbulos crescem para o centro e se atravessam. Não é bug da rotação.
11. `GLUT_DEPTH` ou o bit de limpeza da profundidade esquecidos. A tecla `Z` mente.

12. Parâmetro chamado `near` ou `far` num arquivo que inclui `windows.h`.
13. Julgar a tecla `G` com a luz desligada. Não há o que interpolar.

19 Exercícios

Façam a conta antes de olhar a conferência. Os números que dependem da tecla `M` devem bater com o terminal de outra pessoa, não só com esta página.

1. Em que índice de `float m[16]` fica t_y? E s_y? E o -1 da perspectiva?
2. Com a escala da tecla `M` em 1, sem ter apertado `+`, qual é o ponto $(0{,}20,\ 0,\ 0)$ depois de $R_z(45°)\ T\ S$, em três casas?
3. Apertar `+` uma vez leva a escala a 1,05. Calcule o novo ponto antes de apertar `M`. A última coluna de $R_z T$ muda?
4. Por que a pétala de $+45°$ não coincide com a matriz que `M` imprimiu, mesmo com a escala global em 1?
5. Troquem, no papel, a ordem para $S\ T\ R_z$. Onde fica o centro das quatro pétalas? O que a tecla `+` passa a fazer com a distância ao centro?
6. φ cai de $40°$ para $20°$ e d continua 5,2. A logo cresce ou encolhe? A ponta, a cerca de 0,75 em Y, ainda cabe na meia-altura visível?
7. A escala da pétala é uniforme. Por que `GL_NORMALIZE` ainda muda a luz?
8. A luz é declarada depois de R_y. O que o brilho faz quando o espaço está ligado?
9. Com o Z-buffer desligado, dois fragmentos caem no mesmo pixel. Qual fica?
10. Calcule $v(1/2)$ para o expoente 2,15 pela equação (13). Por que esse anel não está na metade do raio?
11. A normal da seção, no ponto direito da elipse, aponta para o centro. A normal de `glNormal3f` aponta para onde?
12. Qual índice i, qual ângulo e qual cor formam o lóbulo superior esquerdo?

Conferência

1. t_y fica em $m[13]$. s_y fica em $m[5]$. O -1 da perspectiva fica em $m[11]$, última linha e terceira coluna.
2. $(-0{,}170,\ 0{,}453,\ 0{,}000)$. A conta crua dá $(-0{,}1697,\ 0{,}4525,\ 0)$; o `printf` arredonda a três casas.
3. $Sp = (0{,}210,\ 0,\ 0)$, depois da translação $(0{,}210,\ 0{,}440,\ 0)$. $x' = 0{,}707107\,(0{,}210 - 0{,}440) = -0{,}1626$, $y' = 0{,}707107\,(0{,}210 + 0{,}440) = 0{,}4596$. Três casas: $(-0{,}163,\ 0{,}460,\ 0{,}000)$. A coluna da translação não muda, porque T continua $(0,\ 0{,}44,\ 0)$ e R_z é o mesmo. S só alterou o ponto, que não está na origem.
4. A pétala de $+45°$ é a grande: escala 1,10 por cima da escala global, distância $0{,}44 \times 1{,}04 = 0{,}4576$. A tecla `M` usa escala global e distância 0,44. O ângulo coincide; o tamanho e o centro, não.
5. R_z gira a pétala em torno da origem e o centro continua lá. T manda as quatro para $(0,\ d,\ 0)$. S, por fora, escala em torno da origem do mundo, então a distância d cresce junto com o lóbulo. As quatro empilham no $+Y$.
6. Cresce. A meia-altura visível passa de $5{,}2 \tan 20° \approx 1{,}89$ para $5{,}2 \tan 10° \approx 0{,}92$. A ponta em 0,75 ainda cabe.
7. A inversa transposta de uma escala uniforme s multiplica a normal por $1/s$. A direção fica certa e o comprimento deixa de ser 1, então $N \cdot L$ e o expoente especular saem errados. `normalize3` já tinha passado, lá no objeto, antes da modelview.
8. A rotação do objeto entra na lâmpada. O brilho gruda no mesmo ponto da pétala e gira com ela, em vez de caminhar pela superfície.
9. O que foi desenhado por último. No laço, o último é $i = 3$, ângulo 225°, verde, lóbulo grande, inferior direito. Isso vale nos pixels em que essa malha realmente escreveu. Onde ela não cobre, ganha o anterior, e assim por diante.
10. $m(1/2) = 1$, então $v = (1/2)^{2{,}15} \cdot 0{,}38 + (1/2) \cdot 0{,}62$. $(1/2)^{2{,}15} \approx 0{,}226$, e $v \approx 0{,}086 + 0{,}310 = 0{,}396$. A potência puxa para o miolo; o seno devolve parte da folga, não toda.
11. Para fora do fio. No ponto direito, $o(0) = \hat{N}$ aponta para o centro da elipse, que é o lado interno do tubo. $o(\pi) = -\hat{N}$ aponta para o lado externo. Os dois são normais de superfície, cada uma no seu α.
12. Índice 1, ângulo $+45°$, laranja, escala 1,10, distância 0,4576. O centro cai em $x < 0$ e $y > 0$.

20 Mapa do código

Tabela 9: Onde cada assunto mora. A função é o que se cita na arguição; o arquivo é o que se abre.

Assunto	Função	Arquivo
Janela, quadro, teclas	`main`, `display`	`main.c`
Luz e material	`setup_lighting`	`main.c`
Quatro cópias	`draw_petals`	`main.c`
Conta da tecla `M`	`print_matrices`	`main.c`
Produto e layout	`mat4_multiply`	`math3d.c`
Afins	`mat4_make_*`	`math3d.c`
Projeções	`mat4_make_perspective`, `mat4_make_ortho`	`math3d.c`
Porta da OpenGL	`apply_matrix`	`math3d.c`
Curva e triedro	`ellipse_frame`	`petal.c`
Fio	`tube_vertex`, `draw_ring`	`petal.c`
Famílias	`ring_position`, `draw_strand`	`petal.c`
Cabeçalho Windows	macros `near`/`far`	`gl_platform.h`

O que cada pessoa precisa conseguir dizer sem abrir o plano:

- a ordem $R_z T S$, e por que a última chamada age primeiro;

- o índice $m[c \cdot 4 + r]$, com a translação em 12, 13, 14;
- a diferença entre $w = -z$ e $w = 1$;
- a cadeia $T_{\text{cam}} R_x R_y R_z T S$, e o lugar da `glLightfv` dentro dela;
- o tubo: elipse, triedro, $v = p + \rho\, o(\alpha)$, normal igual a o;
- Phong no papel, Blinn na OpenGL, Gouraud na tecla `G`;
- o teste `GL_LESS` contra a ordem de desenho.

21 Glossário

Aspect
Largura da janela dividida pela altura. Entra em f_v/a na perspectiva e na largura do retângulo ortográfico. Sem ele, a elipse estica quando a janela muda de forma.

Blinn–Phong
O modelo de especular que a OpenGL clássica avalia: $(N \cdot H)^n$, com H unitário na direção de $L + V$.

Column-major
A matriz 4×4 armazenada coluna por coluna. É o layout de `glMultMatrixf` e de `m[16]` neste programa.

Coordenada homogênea
O ponto (x, y, z) escrito $(x, y, z, 1)$, de modo que a translação caiba numa matriz.

Divisão perspectiva
Trocar (x, y, z, w) por $(x/w,\ y/w,\ z/w)$. É o passo em que o longe fica menor.

Gouraud
Iluminação nos vértices e interpolação da cor dentro do triângulo. No fonte, `GL_SMOOTH`.

Modelview
A matriz que leva o vértice do objeto ao olho. Aqui ela acumula câmera, yaw, pitch e a cópia da pétala.

Modo imediato
Emitir vértices com `glBegin` e `glEnd`, em vez de armazená-los num buffer.

NDC
Normalized device coordinates. O cubo $[-1, 1]^3$ depois da divisão por w e antes do viewport.

Normal
Vetor perpendicular à superfície. No tubo, é o raio da seção, de comprimento 1. A OpenGL transforma esse vetor pela inversa transposta da modelview.

Pipeline fixo
A OpenGL em que luz, matriz e tonalização são estado da máquina, não shaders.

Projeção
A matriz P que leva o olho ao recorte. Perspectiva ou ortográfica. Não move a câmera.

Triangle strip
Primitiva em que cada vértice novo fecha um triângulo com os dois anteriores. É como cada fatia do tubo viaja ao longo da elipse.

Viewport
O retângulo de pixels que recebe o quadrado $[-1, 1]$ do NDC. No programa, a janela inteira.

Z-buffer
Memória de profundidade por pixel. Com `GL_LESS`, fica quem está mais perto da câmera.

Os números desta apostila foram lidos de `examples/main.c`, `examples/math3d.c` e `examples/petal.c`. Constantes de forma sem nome no fonte — os coeficientes $0{,}058$, $0{,}29$, $0{,}62$, $0{,}72$ — estão nas expressões de `draw_strand` e `ring_position`. Se algum desses literais mudar, a conta da seção correspondente muda com ele; a álgebra, não.